

Tuples and Sets in Python

1. Tuples in Python

A tuple is a collection of items that is **ordered** and **immutable** (unchangeable). Tuples are similar to lists, but once a tuple is created, you cannot modify it. They are often used to group related data together.

Syntax:

```
my_tuple = (element1, element2, element3, ...)
```

Example:

```
my_tuple = ("apple", "banana", "cherry")
```

```
numbers_tuple = (1, 2, 3, 4)
```

Creating a Tuple with One Element:

To create a tuple with only one element, include a trailing comma:

```
single_element_tuple = ("apple",)
```

2. Accessing Tuple Elements

You can access elements in a tuple using **indexing**, just like with lists. Tuples also support negative indexing.

Example:

```
fruits = ("apple", "banana", "cherry")
```

```
print(fruits[0]) # Output: apple
```

```
print(fruits[-1]) # Output: cherry
```

Slicing Tuples:

You can also slice tuples to access a subset of the elements.

```
print(fruits[1:3]) # Output: ('banana', 'cherry')
```

3. Tuple Operations

Although tuples are immutable, you can perform various operations with them.

Tuple Concatenation:

You can combine two or more tuples using the + operator.

```
tuple1 = (1, 2, 3)
```

```
tuple2 = (4, 5, 6)
```

```
combined_tuple = tuple1 + tuple2
```

```
print(combined_tuple) # Output: (1, 2, 3, 4, 5, 6)
```

Tuple Repetition:

You can repeat a tuple multiple times using the `*` operator.

```
repeated_tuple = (1, 2) * 3
```

```
print(repeated_tuple) # Output: (1, 2, 1, 2, 1, 2)
```

Checking Membership:

You can check if an item exists in a tuple using the `in` operator.

```
print("apple" in fruits) # Output: True
```

4. Tuple Methods

Though tuples are immutable, Python provides some built-in methods for working with tuples.

- `count()`: Returns the number of times an element appears in the tuple.
- `my_tuple = (1, 2, 3, 1, 1)`

```
print(my_tuple.count(1)) # Output: 3
```

- `index()`: Returns the index of the first occurrence of an element.
- `my_tuple = ("apple", "banana", "cherry")`

```
print(my_tuple.index("banana")) # Output: 1
```

5. Advantages of Using Tuples

- **Immutable:** This property ensures that tuple data cannot be modified after creation, making them useful for fixed data.
 - **Faster than Lists:** Due to immutability, tuples are generally faster than lists.
 - **Can Be Used as Keys in Dictionaries:** Since tuples are hashable, they can be used as keys in dictionaries, unlike lists.
-

6. Sets in Python

A set is a collection of **unique** items that is **unordered** and **unindexed**. Sets do not allow duplicate values. Sets are useful for performing operations like union, intersection, and difference.

Syntax:

```
my_set = {element1, element2, element3, ...}
```

Example:

```
fruits_set = {"apple", "banana", "cherry"}
```

```
numbers_set = {1, 2, 3, 4, 5}
```

Empty Set:

To create an empty set, use the `set()` function (not `{}`, which creates an empty dictionary).

```
empty_set = set()
```

7. Set Operations

Sets support mathematical operations like union, intersection, and difference.

Union:

The union of two sets combines all elements from both sets, removing duplicates.

```
set1 = {1, 2, 3}
```

```
set2 = {3, 4, 5}
```

```
union_set = set1 | set2 # Output: {1, 2, 3, 4, 5}
```

Intersection:

The intersection of two sets returns elements that are common to both sets.

```
intersection_set = set1 & set2 # Output: {3}
```

Difference:

The difference between two sets returns elements that are in the first set but not in the second.

```
difference_set = set1 - set2 # Output: {1, 2}
```

Symmetric Difference:

The symmetric difference returns elements that are in either of the sets but not in both.

```
sym_diff_set = set1 ^ set2 # Output: {1, 2, 4, 5}
```

8. Set Methods

Sets come with several useful methods for performing common tasks.

- `add()`: Adds an element to the set.
- `fruits_set.add("orange")`

```
print(fruits_set) # Output: {'apple', 'banana', 'cherry', 'orange'}
```

- `remove()`: Removes a specified element from the set. Raises an error if the element does not exist.
- `fruits_set.remove("banana")`

```
print(fruits_set) # Output: {'apple', 'cherry'}
```

- `discard()`: Removes a specified element without raising an error if it does not exist.

```
fruits_set.discard("banana") # No error if "banana" is not in the set
```

- `pop()`: Removes a random element from the set.

```
fruits_set.pop()
```

- `clear()`: Removes all elements from the set.

```
fruits_set.clear()
```

9. Differences Between Lists, Tuples, and Sets

Feature	List	Tuple	Set
Ordering	Ordered	Ordered	Unordered
Mutability	Mutable	Immutable	Mutable
Duplicates	Allows duplicates	Allows duplicates	No duplicates
Indexing	Supports indexing	Supports indexing	No indexing
Operations	List operations	Tuple operations	Set operations
Common Uses	General collection	Fixed data	Unique items